

Lideranças Empáticas — Guia Técnico Completo

Esse documento foi feito majoritariamente por IA, porém, conferido e corrigido pela Equipe

Documento destinado à equipe de desenvolvimento e à apresentação para professores avaliadores.

Explica arquitetura, decisões técnicas, funcionamento de cada componente e pontos críticos do projeto.

Índice

1. [Visão Geral do Projeto](#1-visão-geral-do-projeto)
2. [Arquitetura do Sistema](#2-arquitetura-do-sistema)
3. [Banco de Dados (PostgreSQL)](#3-banco-de-dados-postgresql)
4. [Backend — Server (FastAPI)](#4-backend--server-fastapi)
5. [Dashboard Admin (Dash + Plotly)](#5-dashboard-admin-dash--plotly)
6. [Aplicativo Móvel (Flutter)](#6-aplicativo-móvel-flutter)
7. [Câmera com IA (YOLOv8 + OpenCV)](#7-câmera-com-ia-yolov8--opencv)
8. [Autenticação e Controle de Acesso](#8-autenticação-e-controle-de-acesso)
9. [Deploy com Docker](#9-deploy-com-docker)
10. [Fluxos de Dados Principais](#10-fluxos-de-dados-principais)
11. [Pontos Importantes para Avaliadores](#11-pontos-importantes-para-avaliadores)

1. Visão Geral do Projeto

O **Lideranças Empáticas** é uma plataforma de gestão de doações de alimentos. Grupos (equipes) coletam alimentos e os registram de duas formas:

- **Manual:** via aplicativo móvel, o voluntário informa o tipo e peso do alimento.
- **Automática:** uma câmera com inteligência artificial identifica o alimento em tempo real usando visão computacional.

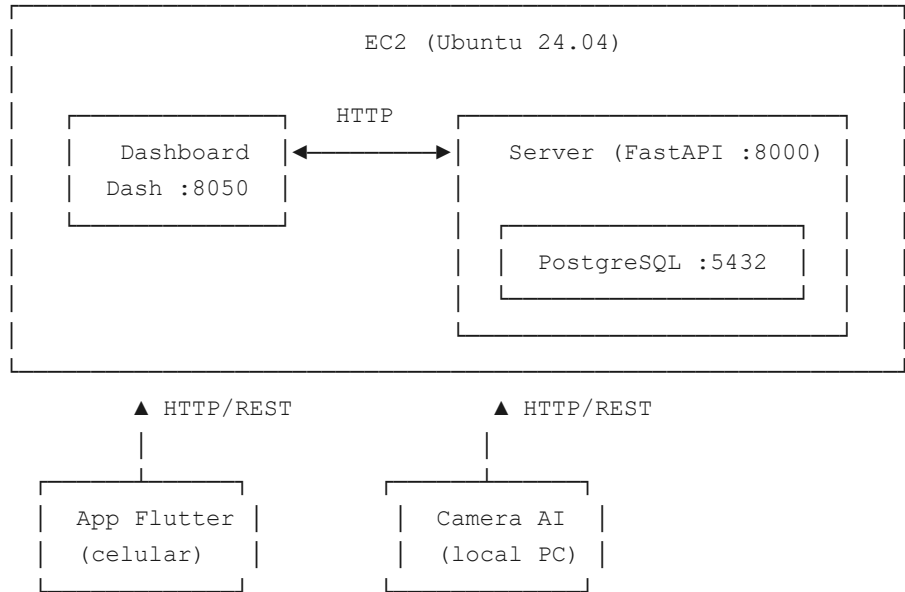
Os dados chegam a um servidor central, que os persiste no banco de dados e os disponibiliza para um painel administrativo (dashboard) onde gestores acompanham o progresso, comparam registros manuais com os automáticos e gerenciam metas por equipe.

Tecnologias por componente

Componente	Linguagem	Framework principal	Porta
Server (API)	Python 3.11	FastAPI + SQLAlchemy	8000
Dashboard (painel)	Python 3.11	Dash + Plotly + Pandas	8050

admin)			
App (celular/web)	Dart	Flutter	— (client)
Camera AI (detecção)	Python 3.11	YOLOv8 + OpenCV	local
Banco de dados	SQL	PostgreSQL 15	5432
Deploy	—	Docker + Docker Compose	—

2. Arquitetura do Sistema



Todos os componentes se comunicam exclusivamente via **API REST HTTP/JSON** com o Server. Não há comunicação direta entre App, Dashboard e Camera AI. O Server é o único ponto que acessa o banco de dados.

3. Banco de Dados (PostgreSQL)

Por que PostgreSQL?

PostgreSQL foi escolhido por ser robusto, open-source, ter suporte excelente a integridade referencial (chaves estrangeiras) e integrar perfeitamente com SQLAlchemy e Docker.

Tabelas e estrutura

teams — Equipes

id SERIAL PRIMARY KEY

```

name          VARCHAR(120) UNIQUE NOT NULL    -- nome único da equipe
active        BOOLEAN DEFAULT TRUE
created_at    TIMESTAMP DEFAULT now()

```

users — *Usuários do sistema*

```

id            SERIAL PRIMARY KEY
name          VARCHAR(120) NOT NULL
email         VARCHAR(120) UNIQUE NOT NULL
password_hash VARCHAR(255) NOT NULL           -- PBKDF2-SHA256, nunca texto
puro
role          VARCHAR(30) NOT NULL             -- 'admin' | 'coordenador' |
'operador'
team_id       INTEGER REFERENCES teams(id) -- NULL para admin
active        BOOLEAN DEFAULT TRUE
created_at    TIMESTAMP DEFAULT now()

```

readings — *Registros manuais (App)*

```

id            SERIAL PRIMARY KEY
team_id       INTEGER NOT NULL REFERENCES teams(id)
user_id       INTEGER NOT NULL REFERENCES users(id)
category      VARCHAR(50) NOT NULL             -- arroz | feijao | macarrao | acucar |
fuba | oleo | outros
kg_amount     FLOAT NOT NULL
created_at    TIMESTAMP DEFAULT now()

```

camera_readings — *Registros automáticos (Câmera IA)*

```

id            SERIAL PRIMARY KEY
team_id       INTEGER NOT NULL REFERENCES teams(id)
category      VARCHAR(50) NOT NULL
confidence     FLOAT NOT NULL                  -- confiança do modelo (0.0 a 1.0)
kg_amount     FLOAT NOT NULL
price         FLOAT NOT NULL
evidence_path  VARCHAR(255)                   -- caminho da foto salva
created_at    TIMESTAMP DEFAULT now()

```

goals — *Metas por equipe/categoria*

```

id            SERIAL PRIMARY KEY
team_id       INTEGER NOT NULL REFERENCES teams(id)
category      VARCHAR(50) NOT NULL
target_kg     FLOAT NOT NULL
created_at    TIMESTAMP DEFAULT now()
UNIQUE(team_id, category)                    -- cada equipe tem no máximo 1 meta por
categoria

```

ORM: SQLAlchemy

O acesso ao banco é feito via **SQLAlchemy** (ORM — Object-Relational Mapper). Isso significa que cada tabela é representada por uma classe Python; as queries são escritas em Python, não em SQL puro. O SQLAlchemy cuida da criação das tabelas, das transações e da conexão com o PostgreSQL. A função `init_db()` é chamada no startup da API e cria as tabelas automaticamente se não existirem (`create_all`).

4. Backend — Server (FastAPI)

Pasta: `SRC/Server/`

Framework: [FastAPI](https://fastapi.tiangolo.com/) — framework Python moderno, assíncrono, com validação automática via Pydantic e documentação automática (Swagger em `/docs`).

Estrutura de arquivos

```
Server/
├── app/
│   ├── main.py                # Ponto de entrada da API
│   ├── models/                # Classes ORM (tabelas do banco)
│   │   ├── user.py
│   │   ├── team.py
│   │   ├── reading.py
│   │   ├── camera_reading.py
│   │   └── goal.py
│   ├── schemas/                # Validação de entrada/saída (Pydantic)
│   │   ├── user.py
│   │   ├── team.py
│   │   ├── reading.py
│   │   ├── camera_reading.py
│   │   └── goal.py
│   ├── routers/                # Endpoints agrupados por recurso
│   │   ├── auth.py
│   │   ├── readings.py
│   │   ├── camera_readings.py
│   │   ├── goals.py
│   │   ├── teams.py
│   │   ├── users.py
│   │   └── public.py
│   ├── core/
│   │   ├── config.py           # Variáveis de ambiente
│   │   └── security.py         # JWT e hashing de senha
│   └── db/
```

```

|         |─ database.py      # Engine + sessão SQLAlchemy
|         |─ base.py        # Importa todos os models
|         |─ init_db.py     # Cria tabelas no startup
|─ Dockerfile
|─ docker-compose.yml
|─ requirements.txt

```

Principais dependências

Biblioteca	Função
<code>fastapi</code>	Framework web (rotas, validação, docs automáticas)
<code>uvicorn[standard]</code>	Servidor ASGI que executa o FastAPI
<code>sqlalchemy</code>	ORM — camada de acesso ao banco de dados
<code>psycopg2-binary</code>	Driver Python para PostgreSQL
<code>pydantic</code>	Validação e serialização dos dados de entrada/saída
<code>python-jose[cryptography]</code>	Geração e validação de tokens JWT
<code>passlib</code>	Hashing seguro de senhas (PBKDF2-SHA256)
<code>python-multipart</code>	Suporte a formulários e upload de arquivos

Endpoints da API

Autenticação (`/api/auth/`)

Método	Rota	Descrição
POST	<code>/api/auth/login</code>	Recebe email+senha, retorna JWT
POST	<code>/api/auth/register</code>	Cria novo usuário
GET	<code>/api/auth/me</code>	Retorna dados do usuário autenticado
POST	<code>/api/auth/refresh</code>	Renova token expirado

Registros manuais (`/api/readings/`)

Método	Rota	Descrição
POST	<code>/api/readings</code>	Cria registro manual (coordenador/operador)
GET	<code>/api/readings</code>	Lista registros (coordenador vê só sua equipe)
GET	<code>/api/readings/summary</code>	Resumo por equipe/categoria

Registros de câmera (`/api/camera-readings/`)

Método	Rota	Descrição
POST	<code>/api/camera-readings</code>	Cria registro da câmera (header <code>X-Camera-Key</code>)
GET	<code>/api/camera-readings</code>	Lista detecções com filtros
GET	<code>/api/camera-</code>	Resumo estatístico

	<code>readings/summary</code>	
--	-------------------------------	--

Metas (`/api/goals/`)

Método	Rota	Descrição
GET	<code>/api/goals</code>	Lista metas (coordenador vê só a sua)
POST	<code>/api/goals</code>	Cria ou atualiza meta (admin, upsert)
DELETE	<code>/api/goals/{id}</code>	Remove meta (admin)

Equipes e Usuários

Método	Rota	Descrição
GET	<code>/api/teams</code>	Lista equipes ativas (público, sem auth)
POST	<code>/api/teams</code>	Cria equipe
GET	<code>/api/users</code>	Lista usuários (admin)
POST	<code>/api/users</code>	Cria usuário (admin)
PUT	<code>/api/users/{id}</code>	Atualiza usuário
DELETE	<code>/api/users/{id}</code>	Remove usuário

Pública (sem autenticação)

Método	Rota	Descrição
GET	<code>/api/public/camera-readings</code>	Leituras da câmera para exibição pública

Como a API valida os dados

O FastAPI usa **Pydantic schemas** para validar automaticamente os dados que chegam. Por exemplo, ao criar uma leitura (`POST /api/readings`), o schema `ReadingCreate` garante que `category` seja uma string não vazia, `kg_amount` seja float positivo, etc. Se os dados estiverem errados, a API retorna `422 Unprocessable Entity` com detalhes do erro — sem nenhum código manual de validação.

Upsert de metas

O endpoint `POST /api/goals` implementa **upsert**: se já existe uma meta para aquela equipe + categoria, ele atualiza o `target_kg`. Se não existe, cria. Isso evita duplicatas e simplifica o uso pelo Dashboard.

5. Dashboard Admin (Dash + Plotly)

Pasta: `SRC/Dashboard/`

Framework: [Plotly Dash](https://dash.plotly.com/) — framework Python para dashboards interativos. Combina um servidor Flask/Werkzeug com componentes React pré-construídos (gráficos Plotly, dropdowns, etc.).

Por que Dash e não React/Vue puro?

Dash permite escrever toda a interface e a lógica de negócio em Python puro, sem JavaScript. Para uma equipe focada em Python e dados, isso reduz drasticamente o tempo de desenvolvimento. Os gráficos são gerados pelo Plotly, biblioteca de visualização de dados líder no ecossistema Python/Data Science.

Estrutura de arquivos

```
Dashboard/
├── app.py          # Inicialização do app Dash, layout raiz, callbacks globais
├── auth.py         # Funções de login (POST ao backend)
├── data.py         # Todas as chamadas HTTP à API do Server
├── layout.py       # Componentes reutilizáveis (sidebar, filtros, KPI boxes)
├── db.py           # Engine SQLAlchemy (consultas diretas opcionais)
├── assets/
│   └── style.css   # CSS customizado (cores, cards, layout responsivo)
└── pages/
    ├── login.py    # Tela de login
    ├── overview.py # Página principal: comparação app vs câmera
    ├── app_data.py # Analytics dos registros manuais
    ├── camera.py   # Analytics das detecções da câmera
    ├── goals.py    # Gestão e visualização de metas
    ├── teams.py    # Gerenciamento de equipes
    └── logout.py   # Encerrar sessão
```

Como o Dash funciona (callbacks)

O Dash usa um modelo **reativo**: componentes têm `id` e os callbacks Python são funções decoradas com `@app.callback` que declaram quais componentes são **Input** e quais são **Output**. Quando o usuário muda um filtro (dropdown de equipe, por exemplo), o Dash chama automaticamente o callback correspondente e atualiza o gráfico. Toda a lógica é no servidor Python — o browser só envia eventos e recebe HTML/JSON atualizado.

Autenticação no Dashboard

O Dashboard não tem banco de dados próprio. O login é feito via chamada HTTP ao backend (`POST /api/auth/login`). O token JWT retornado é armazenado no `dcc.Store` (sessionStorage do browser). Em cada callback que carrega dados protegidos, o token é passado nas chamadas à API. Ao fechar o browser, o token é perdido.

Página de Visão Geral (overview.py)

É a página principal do dashboard. Ela exibe:

- **4 KPIs** no topo: Total kg App, Total kg Câmera, Valor estimado, % da meta atingida
- **Gráfico de barras** agrupado por categoria (App vs Câmera lado a lado)
- **Tabela de comparação** com `diff_kg` (diferença absoluta) e `diff_pct` (diferença percentual)

A lógica de comparação está em `data.py → get_comparison()`: busca as leituras do App e da Câmera, agrupa por categoria e calcula a diferença. Uma divergência muito grande entre App e Câmera indica possível erro de registro manual ou falha da detecção automática.

Página de Metas (goals.py)

Permite ao admin definir metas de coleta (em kg) por equipe e por categoria de alimento. A meta é salva via `POST /api/goals`. O progresso é calculado comparando o total de `camera_readings.kg_amount` da equipe com o `target_kg` da meta.

Principais dependências

Biblioteca	Função
<code>dash</code>	Framework de dashboard interativo
<code>plotly</code>	Biblioteca de gráficos (bar, pie, line, etc.)
<code>pandas</code>	Manipulação de dados em DataFrames
<code>requests</code>	Chamadas HTTP à API do Server
<code>python-dotenv</code>	Carrega variáveis de ambiente do arquivo <code>.env</code>

6. Aplicativo Móvel (Flutter)

Pasta: `SRC/App/`

Framework: [Flutter](https://flutter.dev/) (Dart) — framework Google para apps cross-platform (Android, iOS, Web) a partir de um único código-base.

Por que Flutter?

Flutter compila para código nativo (não WebView), resultando em performance próxima a um app nativo. A equipe precisava suportar Android e potencialmente iOS sem duplicar código.

Estrutura relevante

```
App/lib/
├── core/
│   ├── api/
│   │   ├── api_config.dart      # URL base do servidor
│   │   ├── api_client.dart     # HTTP client com headers de auth
│   │   ├── auth_api.dart       # Login, register, me
│   │   └── readings_api.dart    # CRUD de registros manuais
```



```

|   |   |─ goals_api.dart           # Consulta de metas
|   |   |─ teams_api.dart          # Lista de equipes
|   |   |─ users_api.dart          # Gestão de usuários (admin)
|   |   └─ providers/
|   |       └─ app_provider.dart    # Estado global + enum FoodCategory
|   |   └─ routes/
|   |       └─ app_routes.dart      # Definição de rotas de navegação
|   |   └─ theme/
|   |       └─ app_colors.dart
|   |       └─ app_theme.dart
|   └─ features/auth/
|       └─ models/
|           └─ user_model.dart
|       └─ pages/
|           └─ login_page.dart
|           └─ food_register_page.dart # Formulário de registro manual
|           └─ data_table_page.dart   # Tabela de registros do usuário
|           └─ goals_page.dart        # Visualização de metas
|           └─ admin_dashboard_page.dart
|           └─ coordinator_dashboard_page.dart
|           └─ manage_users_page.dart
|           └─ manage_teams_page.dart
|           └─ manage_goals_page.dart
|           └─ edit_profile_page.dart

```

Categorias de alimentos (FoodCategory)

As categorias são definidas como enum em `app_provider.dart`:

```
enum FoodCategory { arroz, feijao, macarrao, acucar, fuba, oleo, outros }
```

O Dart exige que switches em enums sejam **exaustivos** — todos os casos precisam ser tratados. Isso é uma garantia em tempo de compilação: se uma nova categoria for adicionada ao enum mas esquecida em algum switch, o código não compila.

Gerenciamento de estado

O app usa o pacote **Provider** (`provider: ^6.1.5`). O `AppProvider` é um `ChangeNotifier` que guarda o token JWT, dados do usuário logado e estado global. Os widgets escutam mudanças via `Consumer<AppProvider>` ou `context.watch<AppProvider>()`.

Armazenamento seguro do token

O token JWT é salvo com `flutter_secure_storage`, que usa o Keychain (iOS) ou Keystore (Android) do sistema operacional — não é texto puro em disco.

Principais dependências (pubspec.yaml)

Pacote	Função
http	Requisições HTTP à API
flutter_secure_storage	Armazenamento seguro do token JWT
provider	Gerenciamento de estado global
fl_chart	Gráficos de pizza, barra, linha
path_provider	Acesso a diretórios do sistema

7. Câmera com IA (YOLOv8 + OpenCV)

Pasta: SRC/camera_ai/

Esta é a parte mais técnica do projeto. Combina **visão computacional** (OpenCV), **deep learning** (YOLOv8) e **medição de tamanho físico** (homografia) para identificar e pesar alimentos automaticamente.

Componentes principais

```
camera_ai/
├── detector.py           # Loop principal de detecção (executa diretamente)
├── calibrador.py        # Utilitário de calibração da câmera
├── config.py            # Carrega/salva camera_config.json
├── camera_config.json   # Configuração (URL server, chave API, calibração)
├── best.pt              # Modelo YOLOv8 treinado (pesos)
├── data.yaml            # Configuração do dataset para treino
├── ml.py                # Carregamento lazy do modelo
├── training/
│   ├── train.py         # Script de treinamento YOLOv8
│   ├── generate_dataset.py # Prepara imagens para treino
│   ├── generate_labels.py # Gera arquivos de label YOLO
│   └── split_dataset.py  # Divide dataset em treino/validação
```

O modelo YOLOv8

YOLO (You Only Look Once) é uma família de redes neurais convolucionais para detecção de objetos em tempo real. A versão **YOLOv8n** (nano) foi escolhida por ser leve o suficiente para rodar em CPU comum sem necessidade de GPU dedicada.

O modelo foi **treinado do zero** (fine-tuning) com imagens de alimentos doados. As 7 classes detectadas são:

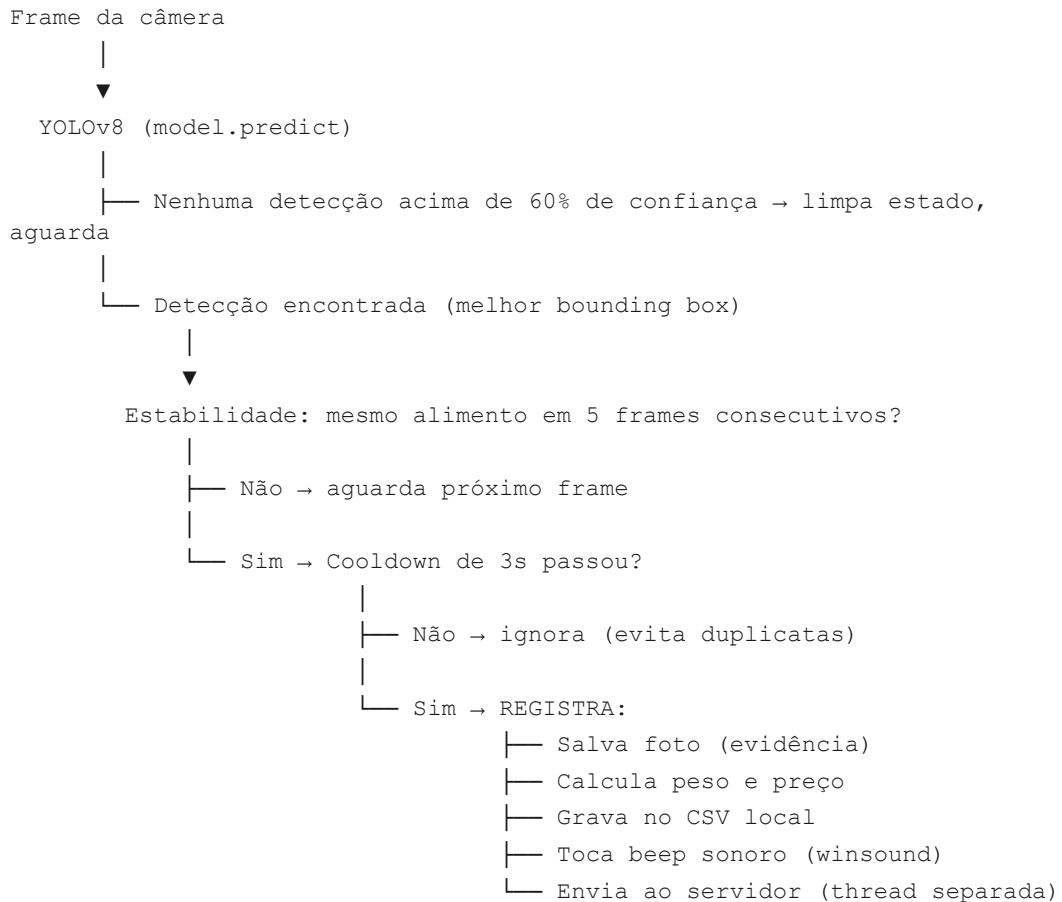
ID	Classe no modelo	Categoria no servidor
0	acucar	acucar
1	arroz	arroz
2	feijao	feijao

3	fuba	fuba
4	macarrao	macarrao
5	misto	outros
6	oleo	oleo

O arquivo `data.yaml` define o caminho do dataset e os nomes das classes para o processo de treino.

Pipeline de detecção (detector.py)

O script roda em loop e segue este fluxo a cada frame:



Por que 5 frames? Para evitar registros acidentais. O modelo pode errar em frames isolados (reflexo de luz, movimento rápido). Exigir 5 frames consecutivos garante que o objeto realmente está na frente da câmera por pelo menos ~0,2 segundos.

Por que cooldown de 3 segundos? Para não registrar múltiplas vezes o mesmo pacote que permanece na frente da câmera.

Mapeamento de categorias (CATEGORY_MAP)

O modelo foi treinado com nomes específicos que podem não coincidir exatamente com as categorias do servidor. O `CATEGORY_MAP` no `detector.py` mapeia variações:

```
CATEGORY_MAP = {
    "arroz": "arroz",
    "feijão": "feijao",    # acentuação
    "misto": "outros",     # classe do modelo → categoria do servidor
    "cafe": "outros",      # café mapeado para outros
    ...
}
```

Calibração de tamanho (Homografia)

Para estimar o **peso real** do pacote, o sistema usa a **transformação de homografia** (técnica de visão computacional):

12. O operador executa `calibrador.py` e clica nos 4 cantos de um objeto de referência de 30×30 cm colocado na esteira.
13. O calibrador calcula a **matriz de homografia** — uma transformação matemática que converte pixels da imagem em centímetros reais.
14. Os pontos de calibração são salvos em `camera_config.json`.
15. No `detector.py`, a largura da bounding box é convertida para cm usando `cv2.perspectiveTransform`.
16. Com base na largura em cm, o código seleciona o peso correspondente (ex: arroz <22cm → 1kg, arroz <27cm → 2kg, arroz ≥27cm → 5kg).

Se a câmera **não estiver calibrada**, o sistema usa pesos padrão fixos (ex: todo arroz = 5kg). A linha amarela desenhada na base do objeto na tela indica que a calibração está ativa.

Envio ao servidor em thread separada

O envio HTTP (`POST /api/camera-readings`) é feito em uma `Thread` separada para não bloquear o loop de vídeo. A câmera continua capturando frames enquanto o request HTTP acontece em paralelo. O status do envio (`pendente` → `enviado` ou `erro`) é atualizado no CSV local.

Autenticação da câmera

A câmera **não usa JWT** para enviar registros. Ela usa uma **chave de API fixa** no header `X-Camera-Key: camera-secret-key`. Isso simplifica a integração: a câmera não precisa fazer login nem renovar tokens. O servidor valida essa chave em `camera_readings.py` antes de aceitar o registro.

Beep sonoro

Quando um alimento é registrado com sucesso, o sistema toca um beep usando `winsound.Beep(1000, 220)` — 1000 Hz por 220ms. O `winsound` é uma biblioteca **nativa do Windows**, não requer instalação. O beep roda em uma thread daemon separada para não afetar o vídeo.

Principais dependências

Biblioteca	Função
<code>ultralytics</code>	YOLOv8 — detecção de objetos
<code>opencv-python</code>	Captura de câmera, desenho na tela, homografia
<code>requests</code>	Envio de dados ao servidor
<code>numpy</code>	Operações matriciais (homografia)
<code>winsound</code>	Sinal sonoro (nativo Windows)

8. Autenticação e Controle de Acesso

JWT (JSON Web Token)

Após o login, o servidor gera um **JWT** assinado com uma chave secreta (HMAC-SHA256). O token contém:

```
{
  "sub": "1",                // ID do usuário
  "email": "user@ex.com",
  "role": "admin",
  "name": "Admin",
  "team_id": null,
  "exp": 1234567890         // expiração (Unix timestamp)
}
```

O token é enviado em todas as requisições protegidas no header:

```
Authorization: Bearer <token>
```

O servidor valida a assinatura e a expiração a cada request — sem consultar o banco de dados, o que torna a autenticação muito rápida.

Hierarquia de papéis (roles)

Role	O que pode fazer
admin	Tudo: criar usuários, equipes, metas, ver todos os dados
coordenador	Ver e registrar dados apenas da própria equipe
operador	Apenas criar registros manuais para a própria

	equipe
--	--------

Como o controle de acesso é aplicado na API

Cada router extrai o role do token JWT e aplica restrições:

```
# Exemplo em readings.py
if role == "coordenador":
    q = q.filter(Reading.team_id == user_team_id) # filtra só a equipe
elif role == "admin" and team_id:
    q = q.filter(Reading.team_id == team_id)      # admin pode filtrar
qualquer equipe
```

A função `_require_admin()` lança HTTP 403 se o token não for de admin.

9. Deploy com Docker

Por que Docker?

Docker garante que o ambiente de execução seja **idêntico** entre desenvolvimento local e produção (EC2). Elimina o problema de "funciona na minha máquina" e simplifica o processo de start: um único comando sobe tudo.

Serviços no docker-compose.yml

```
services:
  db:          # PostgreSQL 15 - banco de dados
  backend:     # FastAPI - API REST
  dashboard:   # Dash - painel administrativo
```

Todos os serviços compartilham uma **rede interna Docker** (`appnet`). O Dashboard chama o backend via `http://liderancas_backend:8000` (nome do container, não IP). O PostgreSQL não fica exposto na internet — apenas o backend acessa a porta 5432 internamente.

Variáveis de ambiente (.env)

```
POSTGRES_DB=exemplo_db
POSTGRES_USER= exemplo_user
POSTGRES_PASSWORD= exemplo_de_senha_segura
DATABASE_URL=postgresql:// exemplo:pass@db:5432/ exemplo_db
SECRET_KEY=chave_exemplo_jwt_super_secreta
CAMERA_API_KEY=chave_da_api_camera_secreta
```

O arquivo `.env` **nunca é commitado** no Git (está no `.gitignore`). Cada ambiente (local, produção) tem seu próprio `.env`.

restart: always

Os serviços `backend` e `dashboard` têm `restart: always` no compose. Isso significa que o Docker reinicia automaticamente o container se ele cair por qualquer motivo (erro, reinicialização do servidor EC2, etc.). Garante disponibilidade contínua.

Servidor de produção

O sistema está rodando em uma instância **AWS EC2** (Ubuntu 24.04 LTS, t3.micro, 20GB gp3) no IP público `18.234.178.16`. O EC2 tem 2GB de swap configurado para aliviar a memória RAM de 1GB do t3.micro. O acesso externo é via HTTP nas portas 8000 (API) e 8050 (Dashboard).

10. Fluxos de Dados Principais

Fluxo 1: Registro manual (App)

1. Voluntário abre o app Flutter e faz login
2. App POST `/api/auth/login` → recebe JWT
3. Voluntário seleciona categoria e peso → POST `/api/readings` (Bearer token)
4. Server valida token, valida categoria, salva na tabela `readings`
5. Dashboard atualiza ao abrir a página (GET `/api/readings`)

Fluxo 2: Detecção automática (Câmera)

1. Operador roda `detector.py` na máquina com câmera conectada
2. YOLOv8 detecta alimento em 5 frames consecutivos
3. Sistema calcula peso via homografia (ou peso padrão)
4. Salva foto como evidência local
5. Thread POST `/api/camera-readings` (X-Camera-Key header)
6. Server valida chave, valida categoria, salva em `camera_readings`
7. Dashboard exibe na página "Câmera YOLO"

Fluxo 3: Acompanhamento de metas

1. Admin acessa Dashboard → página Metas
 2. Define meta: equipe X deve coletar Y kg de arroz
 3. Dashboard POST `/api/goals` → Server salva (upsert por `team_id` + `category`)
 4. Dashboard GET `/api/camera-readings` → soma `kg_amount` por categoria/equipe
 5. Calcula progresso: $(\text{kg coletado} / \text{target_kg}) \times 100\%$
 6. Exibe barra de progresso visual
-

11. Pontos Importantes para Avaliadores

Questões técnicas prováveis e respostas

"Por que usaram YOLOv8 e não outra rede neural?"

YOLOv8 é o estado da arte em detecção de objetos em tempo real para hardware comum. É open-source (Ultralytics), tem excelente documentação, suporte ativo e atinge boa precisão mesmo com datasets pequenos via transfer learning. A versão **nano** (yolov8n) roda em CPU sem GPU dedicada, viabilizando o deploy em computadores comuns.

"Como garantem que o peso está correto?"

Dois mecanismos:

17. **Calibração por homografia:** a câmera é calibrada com um objeto de referência de tamanho conhecido (30×30 cm). A matriz de homografia converte pixels em cm reais, e a largura do pacote determina o peso (ex: arroz <22cm = 1kg, <27cm = 2kg, ≥27cm = 5kg).
18. **Peso padrão:** quando não há calibração, usamos pesos fixos por categoria baseados nos tamanhos mais comuns de embalagem no mercado brasileiro.

"Como evitam registros duplicados?"

Dois controles combinados:

- **Estabilidade de frames:** exige 5 frames consecutivos com o mesmo alimento antes de registrar.
- **Cooldown:** após um registro, o mesmo alimento só é registrado novamente após 3 segundos. Isso evita contar o mesmo pacote múltiplas vezes enquanto está parado na frente da câmera.

"Como funciona a autenticação? É segura?"

- Senhas armazenadas com **PBKDF2-SHA256** (nunca texto puro).
- Autenticação via **JWT** com expiração de 24h, assinado com chave secreta do servidor.
- A câmera usa **chave de API** fixa — separada do JWT de usuários para não expor credenciais de login.
- HTTPS não está configurado neste ambiente de desenvolvimento/avaliação, mas seria necessário em produção.

"Por que o Dashboard é separado do Server?"

Separação de responsabilidades (SoC): o Server é a fonte da verdade (dados), o Dashboard é apenas um consumidor que visualiza esses dados via API. Essa arquitetura permite que o App mobile e o Dashboard usem exatamente a mesma API, garantindo consistência. Também permite escalar os serviços independentemente.

"Como funciona o controle de acesso por papel (role)?"

O role do usuário é embutido no JWT no momento do login. Cada endpoint da API extrai o role do token e decide o que mostrar: `coordenador` vê só dados da própria equipe, `admin` vê tudo. Não é possível falsificar o role sem a chave secreta JWT do servidor.

"O sistema funciona offline?"

O App consegue usar a interface, mas os registros precisam ser enviados ao servidor para serem válidos. A câmera salva um CSV local e uma foto de evidência — se o servidor estiver fora do ar, os dados ficam no CSV e podem ser reconciliados manualmente. Não há sincronização automática offline.

"Como foi feito o treinamento do modelo?"

O dataset foi criado com imagens reais dos alimentos (arroz, feijão, macarrão, açúcar, fubá, óleo). As imagens foram anotadas (bounding boxes) com as classes correspondentes no formato YOLO (arquivo `.txt` por imagem). O script `training/train.py` usa transfer learning a partir do `yolov8n.pt` (pesos pré-treinados no ImageNet/COCO) e refina para as 7 classes do projeto.

"Por que o banco de dados não tem migrações (Alembic)?"

Para este escopo de projeto, usamos `create_all()` do SQLAlchemy que cria as tabelas automaticamente no startup se não existirem. Alembic seria recomendado em produção para gerenciar mudanças incrementais no schema sem perda de dados.

"Como o Dashboard se autentica?"

O Dashboard faz login como usuário via `POST /api/auth/login`, recebe um JWT e o armazena na `sessionStorage` do browser (`dcc.Store` do Dash). Cada chamada à API inclui esse token no header `Authorization: Bearer`. O token não persiste após fechar o browser.

"O que é o endpoint público (/api/public)?"

É uma rota sem autenticação que expõe dados agregados das doações para uso em painéis públicos (ex: tela no local da coleta mostrando o total arrecadado). Não expõe informações sensíveis (nomes de usuários, dados financeiros).

Diferenciais do projeto

- **Dupla fonte de dados** com comparação integrada (manual vs automático) — isso detecta discrepâncias e aumenta a confiabilidade dos dados.
- **Calibração física real** para estimativa de peso — não é só classificação visual, tem uma base metrológica.
- **Beep sonoro** como feedback imediato ao operador da câmera — ergonomia operacional.
- **Foto de evidência** salva para cada detecção — auditabilidade e rastreabilidade.

- **Roles granulares** (admin / coordenador / operador) — cada nível acessa apenas o que precisa.
- **Deploy containerizado** com `restart: always` — disponibilidade contínua sem intervenção manual.